

## Apunte de Cátedra

# ALGORITMOS DE BUSQUEDA

## 1.- INTRODUCCION

Buscar en un arreglo un determinado valor significa localizar un elemento del arreglo cuyo contenido coincida con él. Una vez finalizada la búsqueda puede suceder:

- que la búsqueda haya tenido éxito, habiendo localizado la posición donde estaba almacenado el elemento buscado, o
- que la búsqueda no haya tenido éxito, concluyéndose que no existía ningún elemento a buscar.

Las búsquedas son independientes del estado del arreglo. Es evidente que se puede buscar en arreglos ordenados como en arreglos desordenados, utilizando diferentes algoritmos de búsqueda.

Antes de pasar a desarrollar los principales algoritmos, es necesario aclarar que todos los métodos presentados van incluidos en la siguiente clase:

```
public final class Searches  
{  
}
```

## 2.- ALGORITMOS BASICOS DE BUSQUEDA

### 2.1.- Búsqueda Secuencial

Es la forma más simple de buscar un elemento y consiste en examinar secuencialmente uno a uno hasta encontrar el elemento buscado o haber revisado todos los elementos sin éxito. Si tenemos un arreglo de enteros y un elemento que tratamos de localizar, un algoritmo simple que devuelve la posición del elemento en el arreglo, es éste se encuentra en el mismo, o -1 si no se encuentra, es el siguiente:

```
public static int sequentialSearch(int[] vec, int search)  
{  
    int pos=0;  
    while(pos<vec.length)  
    {  
        if(vec[pos]==search)  
            return pos;  
        pos++;  
    }  
    return -1;  
}
```

En el caso de que pudiese haber 2 o más ocurrencias del mismo valor, se encuentra la primera de ellas. Sin embargo, es posible modificar el algoritmo para obtener todas las ocurrencias del dato buscado.

Para estudiar su complejidad, vamos a estudiar los casos mejor, peor y medio de la llamada al método *sequentialSearch(int[], int)*.

- En el mejor caso el algoritmo de búsqueda Secuencial termina tan pronto como encuentra el elemento buscado en el arreglo. Si tenemos mucha suerte, puede ser que la primera posición examinada contenga el elemento que buscamos, en cuyo caso el algoritmo informará que tuvo éxito después de una sola comparación. Por lo tanto, su complejidad será  $O(1)$ .

- En el peor caso sucede cuando encontramos el elemento a buscar en la última posición del arreglo o cuando no se encuentra el elemento en el arreglo. Como se requieren  $n$  ejecuciones del bucle mientras, la cantidad de tiempo es proporcional a la longitud del arreglo  $n$ , más un cierto tiempo para realizar las condiciones del bucle mientras y para la llamada al método. Por lo tanto, la cantidad de tiempo es de la forma  $an + b$  para ciertas constantes  $a$  y  $b$ . En notación  $O$ ,  $O(an + b) = O(an) = O(n)$ .

## Apunte de Cátedra

- En el caso medio, supongamos que cada elemento almacenado en el arreglo es igualmente probable que sea buscado. La media se puede calcular tomando el tiempo total de encontrar todos los elementos y dividiéndolo por n:

$$\text{Total} = a(1 + 2 + \dots + n) + bn = a(n(n + 1) / 2) + bn$$

$$\text{Media} = (\text{Total} / n) = a((n + 1) / 2) + b \text{ que es } O(n).$$

### 2.2.- Búsqueda Binaria

Si los elementos sobre los que se realiza la búsqueda están ordenados, entonces podemos utilizar un algoritmo de búsqueda mucho más rápido que el Secuencial, la búsqueda Binaria. Consiste en reducir paulatinamente el ámbito de búsqueda a la mitad de los elementos, basándose en comparar el elemento a buscar con el elemento que se encuentra en la mitad del intervalo y en base a esta comparación:

1. Si el elemento buscado es **menor** que el elemento medio, entonces sabemos que el elemento está en la mitad inferior del arreglo.
2. Si es **mayor** es porque el elemento está en la mitad superior.
3. Si es **igual** se finaliza con éxito la búsqueda ya que se ha encontrado el elemento.

La implementación de este algoritmo, se muestra a continuación:

```
public static int binarySearch(int[] vec, int search)
{
    int first=0;
    int mid;
    int last=vec.length - 1;
    while(first<=last)
    {
        mid=(first + last) / 2;
        if(vec[mid]==search)
            return mid;
        else
            if(vec[mid]>search)
                last=mid - 1;
            else
                first=mid + 1;
    }
    return -1;
}
```

Para poder medir la velocidad de cálculo del algoritmo de búsqueda Binaria, se deberán obtener el número de comparaciones que realiza el algoritmo, es decir, el número de vueltas del ciclo o el número de recursiones. Aunque en principio puede parecer que ambas versiones invierten el mismo tiempo, la recursiva es más lenta a medida que se incrementa el número de elementos, ya que existirán más llamadas a la función por resolver, con el consiguiente gasto de tiempo de guardar y restaurar parámetros.

En el mejor caso, la búsqueda Binaria podría toparse con el elemento buscado en el primer punto medio, requiriéndose sólo una comparación de elementos. Esto equivale al caso óptimo durante una búsqueda Secuencial, pero en el peor de los casos la búsqueda Binaria es mucho más rápida cuando N es grande.

El algoritmo de búsqueda Binaria progresivamente va disminuyendo el número de elementos sobre el que realizar la búsqueda a la mitad: n, n/2, n/4, ... Así, tras logn divisiones se habrá localizado el elemento o se tendrá la seguridad de que no estaba.

- En el mejor caso, en sus casos óptimos, tanto la búsqueda Secuencial como la Binaria requieren sólo una comparación; esto significa que sus tiempos de ejecución óptimos no dependen de la cantidad de datos: son constantes y por lo tanto, proporcionales a 1, es decir, son de O(1).

- En el peor caso, la búsqueda Secuencial y la Binaria sí dependen de N. La primera, recorre todo el arreglo, requiriendo un tiempo de O(n); la Binaria divide el arreglo, requiriendo sólo un tiempo O(logn). Lo mismo ocurre para el caso medio.

## Apunte de Cátedra

### 3.- ALGORITMOS AVANZADOS DE BUSQUEDA

#### 3.1.- Búsqueda por Interpolación

Este algoritmo procede igual que la búsqueda Binaria sólo que el arreglo se va dividiendo acorde a nuestra estimación en dónde el elemento a buscar se encuentra ubicado. Dada una distribución uniforme de elementos, la búsqueda por Interpolación tiene un caso medio de complejidad  $O(\log \log n)$ .

```
public static int interpolationSearch(int[] vec, int search)
{
    int first=0;
    int mid;
    int last=vec.length - 1;
    while(search >= vec[first] & search <= vec[last])
    {
        mid = first + (int) Math.abs(Math.floor((search - vec[first]) * (last - first) / (vec[last] - vec[first])));
        if(search == vec[mid])
            return mid;
        else
        {
            if(search < vec[mid])
                last = mid - 1;
            else
                first = mid + 1;
        }
    }
    return -1;
}
```

Si las siguientes condiciones son verdaderas, entonces la búsqueda por Interpolación puede ser mejor que la búsqueda Binaria:

- Cada acceso es muy costoso comparado a la instrucción típica, por ejemplo, el arreglo se encuentra almacenado en el disco y cada comparación requiere un acceso a disco.
- Los datos no sólo se encuentran ordenados sino que también se encuentran medianamente distribuidos uniformemente, por ejemplo, una guía telefónica cumple esas características, en cambio una entrada [1, 2, 3, 4, 5, 6, 7, 8, 16, 32, 355, 1000, 12300...] no.

En esta situación tenemos la intención de gastar más tiempo para hacer un cálculo acertado donde el elemento puede ser (en vez de elegir siempre el punto medio), por ejemplo:

- Un arreglo de 1000 elementos.
- El elemento más pequeño del arreglo es 1.000.
- El elemento más grande del arreglo es 1.000.000.
- Buscamos el elemento con el valor 12.000.
- Luego esperamos encontrar el elemento, más o menos, en la posición 11 (siempre asumiendo que los elementos están uniformemente distribuidos). Esto se expresa con la fórmula:

|                 |     |       |        |           |
|-----------------|-----|-------|--------|-----------|
| n = 1.000       | vec | 1.000 | ... .. | 1.000.000 |
| search = 12.000 |     | 0     | ?      | 999       |
|                 |     | ↑     | ↑      | ↑         |
|                 |     | first | mid    | last      |

**Búsqueda Binaria:**  $mid = \frac{(first + last)}{2}$ ;  $mid = \frac{(0 + 999)}{2} = 500$

**Búsqueda por Interpolación:**  $mid = First + \frac{(search - vec[first]) * (last - first)}{vec[last] - vec[first]}$ ;

$$mid = 0 + \frac{(12.000 - 1.000) * (999 - 0)}{1.000.000 - 1.000} = \frac{10.989.000}{999.000} = 11$$

## Apunte de Cátedra

En el análisis, podemos concluir lo siguiente:

- El cálculo es más costoso que el cálculo de la búsqueda Binaria.
- Una iteración puede ser más lenta que la búsqueda Binaria completa.
- Si el costo de estos cálculos es insignificante para el costo de acceso a un elemento, sólo nos interesa el número de iteraciones.
- En el peor caso, cuando los números no se encuentran uniformemente distribuidos, el tiempo de ejecución puede ser lineal y todos los elementos podrían ser examinados.
- Si los elementos son de manera razonable uniformemente distribuidos, el tiempo de ejecución ha sido demostrado como  $O(\log \log n)$  (aplicar el logaritmo dos veces en la sucesión). Por ejemplo, para  $n=4$  billones,  $\log n$  es alrededor 32 y  $\log \log n$  es aproximadamente 5.

### 3.2.- Búsqueda Fibonacci

En vez de dividir el arreglo por la mitad (búsqueda Binaria), esta implementación divide el arreglo según los números de Fibonacci. Para substraer el número Fibonacci disminuye el tamaño del arreglo de disminuciones. Los números de Fibonacci se definen de la siguiente manera:

$$\begin{aligned}F(0) &= 0 \\F(1) &= 1 \\F(n) &= F(n - 1) + F(n - 2) \text{ (para } n \geq 2)\end{aligned}$$

Veamos el código correspondiente a la búsqueda Fibonacci:

```
public static int fibonacciSearch(int[] vec, int search)
{
    //for(j=1; fib(j)<n; j++)
    int f1=1;
    int f2=1;
    int faux;
    int mid;
    int length=vec.length - 1;
    while(f1<length)
    {
        //encuentra a f1 tal que f1>=length
        f1=f1 + f2; //siguiente número Fibonacci
        f2=f1 - f2; //guarda al f1 anterior
    }
    f1=f1 - f2; //encuentra el número Fibonacci más chico
    f2=f2 - f1; //f1=fib(j-2), f2=fib(j-3)
    mid=length - f1 + 1;
    while(search!=vec[mid]) //si no lo encuentra
    {
        if(mid<0 | search>vec[mid])
        {
            //busca en la mitad más baja
            if(f1==1) //si no lo encuentra retorna -1
                return -1;
            mid=mid + f2; //disminuye los números de Fibonacci
            f1=f1 - f2;
            f2=f2 - f1;
        }
        else
        {
            //busca en la mitad más alta
            if(f2==0) //si no lo encuentra retorna -1
                return -1;
            mid=mid - f2; //disminuye los números de Fibonacci
            faux=f1 - f2; //esta vez, disminuye mas
            f1=f2; //para el arreglo más chico
            f2=faux;
        }
    }
    return mid;
}
```

## Apunte de Cátedra

---

El algoritmo de búsqueda de Fibonacci divide el rango de búsqueda en dos partes, de acuerdo a los números de Fibonacci, luego se compara el elemento a buscar con la posición  $F(n - 2)$ . Si el elemento es más chico, continuamos en la mitad más baja del arreglo y en caso contrario, con la mitad más alta del arreglo.

Al igual que la búsqueda Binaria, en el peor caso, el tiempo de ejecución de la búsqueda Fibonacci es de  $O(\log n)$ .

### 4.- HASHING

Existe otro método que puede aumentar la velocidad de búsqueda donde los datos no necesitan estar ordenados y esencialmente es independiente del número  $n$ . Este método se conoce como transformación de claves (clave-dirección) o hashing. El hashing consiste en convertir el elemento almacenado (numérico o alfanumérico) en una dirección (índice) dentro del arreglo.

El objetivo del *direccionamiento hash* se presentará mediante el siguiente ejemplo:

Supongamos una compañía que maneja un inventario consistente de 100 artículos, donde cada artículo tiene un número identificador único de dos dígitos. La forma más obvia de almacenar esta información es declarando un arreglo del tipo:

TArticulo posee los atributos:

Número artículo;

Descripción artículo;

TArticulo [ ] Inventario=new TArticulo [100]; //arreglo de 100 elementos.

donde Inventario[i] representa el registro cuyo número de artículo es  $i$ . Los números de los artículos son las claves utilizadas como índices del arreglo.

Incluso si la compañía almacena menos de 100 artículos, se puede utilizar la misma estructura para mantener el arreglo (siempre y cuando las claves sigan siendo de dos dígitos). Aun cuando muchas posiciones en Inventario estén vacías, esta pérdida es compensada con la ventaja de los accesos directos a cada uno de los artículos existentes en el arreglo Inventario.

Pero este sistema no es siempre práctico. Supongamos que la compañía tiene un inventario de no más de mil artículos, y que el Número de artículo es un número de siete dígitos para cada artículo. Para seguir utilizando direccionamiento directo se requeriría un arreglo de 10 millones de elementos. Esta pérdida de espacio es inaceptable, ya que es muy poco probable que la compañía almacene más de mil artículos. Haría falta algún método de convertir una clave dentro de un rango limitado, de manera que idealmente no hubiera dos claves convertidas al mismo entero.

Siguiendo con el ejemplo y suponiendo que la compañía tiene menos de mil artículos, y que existe solamente un registro por cada artículo, está claro que con un arreglo de 1000 elementos es suficiente para contener toda esa información. Si el arreglo está indexado mediante un entero entre 0 y 999 (ambos inclusive), se pueden utilizar los últimos tres dígitos del número de artículo como índice para el registro del artículo correspondiente en el arreglo.

Utilizando esta técnica, dos claves que están relativamente muy cerca por número de artículo, como 4618396 (Inventario[396]) y 4618996 (Inventario[996]), pueden estar muy separadas la una de la otra en el arreglo, comparadas con dos claves que están separadas numéricamente como 0000991 (Inventario[991]) y 9846995 (Inventario[995]). Esto es debido a que sólo los últimos tres dígitos de la clave se utilizan para determinar la posición del registro en el arreglo.

La idea general de usar la clave para determinar la dirección del registro es una excelente idea, pero se debe modificar de forma que no se desperdicie tanto espacio. Esta modificación se lleva a cabo mediante una función que transforma una clave en un índice de un arreglo y que se denomina *función de Randomización o Hash*. Si  $H$  es una función hash y  $X$  es un elemento a almacenar, entonces  $H(X)$  es la función hash del elemento y se corresponde con el índice donde se debe colocar  $X$ . En nuestro ejemplo, la función hash sería  $H(X) = X \% 1000$  (función resto).

## Apunte de Cátedra

Los valores generados por H deben cubrir todo el conjunto de índices del arreglo. Además, el tamaño del arreglo debe ser un poco más grande que el número de elementos que han de ser insertados, aunque queden posiciones del arreglo sin uso.

El método anterior tiene una deficiencia: suponer que dos elementos X e Y son tales que  $H(X) = H(Y)$ . Entonces, cuando un elemento X entra en el arreglo, éste se inserta en la posición dada por su función Hash,  $H(X)$ . Pero cuando al elemento Y se le asigna su posición donde va a ser insertado mediante la función hash, resulta que la posición que se obtiene es la misma que la del elemento X. Esta situación se denomina *Randomización o Hashing con colisión o choque*.

En el arreglo Inventario se produciría una colisión, por ejemplo, si después de haber insertado en el arreglo el elemento con Número de artículo 4957397, al cual le corresponde la posición 397, se intenta insertar en el arreglo el elemento 0596397, pues la función hash le vuelve a asignar la misma posición 397 a este elemento, y esa posición ya está ocupada.

Una buena función Hash será aquella que:

- 1) minimice los choques o coincidencias, y;
- 2) distribuya los elementos uniformemente a través del arreglo.

Esta es la razón por la que el tamaño del arreglo debe ser un poco mayor que el número real de elementos a insertar, pues cuanto más grande sea el rango de la función de randomización, es menos probable que dos claves generen el mismo valor de asignación o hash, es decir, que se asigne una misma posición a más de un elemento.

Habrà que llegar a un compromiso entre *Eficiencia en Espacio-Tiempo*: el dejar espacios vacíos en el arreglo es una deficiencia en cuanto a espacio, mientras que reduce la necesidad de resolver los casos de choque en la asignación, y por lo tanto es más eficiente en términos de tiempo.

### 4.1.- Métodos de transformación de claves

Los dos criterios principales al seleccionar una función hash H son:

- H deber ser muy fácil y rápida de calcular.
- H debe, en la medida de lo posible, distribuir uniformemente las direcciones hash sobre el conjunto de direcciones del arreglo de forma que haya el menor número de colisiones posible.

Naturalmente, no existe garantía de que la segunda condición se pueda cumplir plenamente sin conocer de antemano las claves y las direcciones. Sin embargo, ciertas técnicas generales prestan una gran ayuda. Una de estas técnicas es la de “trocear” una clave X en pedazos y combinarlos de alguna forma para obtener la dirección hash  $H(X)$ . (El término “hash” viene - en inglés - de esta técnica de “trocear” una clave en pedazos). A continuación exponemos algunas funciones hash de uso más extendidas.

#### 4.1.1.- Restas sucesivas

Esta función se emplea con claves numéricas entre las que existen huecos de tamaño conocido, obteniéndose direcciones consecutivas. Por ejemplo, si el número de expediente de un alumno universitario está formado por el año de entrada en la universidad, seguido de un número identificativo de tres cifras, y suponiendo que entran un máximo de 400 alumnos al año, se le asignarían las claves:

1998-000 --> 0 = 1998000 - 1998000  
1998-001 --> 1 = 1998001 - 1998000  
1998-002 --> 2 = 1998002 - 1998000  
...  
1998-399 --> 399 = 1998399 - 1998000  
1999-000 --> 400 = 1999000 - 1999000 + 400  
...  
yyyy-nnn --> N = yyyynnn - yyyy000 + (400 \* (yyyy-1998))

## Apunte de Cátedra

---

### 4.1.2.- Método de división o resto

Se escoge un número  $m$  menor o igual que la capacidad  $n$  de elementos.

La función hash  $H$  se define por:

$$H(X) = X \% m \quad \text{o} \quad H(X) = (X \% m) + 1$$

donde  $X \% m$  indica el resto de la división de  $X$  por  $m$ . La segunda fórmula se usa cuando queremos que las direcciones hash vayan de  $1$  a  $m$  en vez de desde  $0$  hasta  $m-1$ .

El mejor resultado del método de división se obtiene cuando  $m$  es primo (es decir,  $m$  no es divisible por ningún entero positivo distinto de  $1$  y  $m$ ).

Si el número  $m$  es el  $13$ , los números siguientes quedan transformados en:

13000000 -->  $0 = 13000000 \% 13$   
12345678 -->  $7 = 12345678 \% 13$   
13602499 -->  $1 = 13602499 \% 13$   
71140205 -->  $6 = 71140205 \% 13$   
73062137 -->  $5 = 73062137 \% 13$

Este método presenta problemas de colisión que veremos más adelante.

### 4.1.3.- Método del medio cuadrado

La clave es multiplicada por sí misma y los dígitos del medio (el número exacto depende del rango del índice) del cuadrado son utilizados como índice.

Es importante que se usen las mismas posiciones del cuadrado para todas las claves.

123 \* 123 = 15129 --> 51  
136 \* 136 = 18496 --> 84  
730 \* 730 = 532900 --> 29  
301 \* 301 = 90601 --> 06  
625 \* 625 = 390625 --> 06

Este método también presenta problemas de colisión.

### 4.1.4.- Truncamiento

Ignora parte de la clave y se utiliza la parte restante directamente como índice. Si las claves, por ejemplo, son enteros de  $8$  dígitos y el arreglo de transformación tiene  $1000$  posiciones, entonces el  $1^\circ$ ,  $3^\circ$  y  $8^\circ$  dígitos pueden formar la función hash. Por ejemplo,  $72588495$  se convierte en  $755$ .

El truncamiento es un método muy rápido, pero falla para distribuir las claves de modo uniforme. También presenta problemas de colisión.

### 4.1.5.- Método de superposición

Consiste en la división de la clave en diferentes partes y su combinación en un modo conveniente (a menudo utilizando suma o multiplicación) para obtener el índice. La clave  $X$  se divide en varias partes  $X_1, X_2, \dots, X_n$ , donde cada una, con la única posible excepción de la última, tiene el mismo número de dígitos que la dirección especificada. A continuación, se suman todas las partes. En esta operación se desprecian los dígitos más significativos que se obtengan de arrastre o acarreo. También presenta problemas de colisión.

Hay dos formas de conseguir la función hash mediante este método:

1. *superposición por desplazamiento* donde todas las partes se suman entre sí.

13000000 -->  $130 = 130 + 000 + 00$   
12345678 -->  $657 = 123 + 456 + 78$   
71140205 -->  $118 = 1118 = 711 + 402 + 05$   
13602499 -->  $259 = 136 + 024 + 99$   
35000010 -->  $360 = 350 + 000 + 10$

## Apunte de Cátedra

---

2. *superposición por plegamiento* se hace la inversa a las partes pares  $X_2, X_4, \dots$ , antes de sumarlas, con el fin de afinar más.

13000000 --> 130 = 130 + 000 + 00  
12345678 --> 855 = 123 + 654 + 78  
71140205 --> 920 = 711 + 204 + 05  
13602499 --> 655 = 136 + 420 + 99  
35000010 --> 359 = 350 + 000 + 10

Las claves no tienen por qué ser numéricas y en ellas podrán aparecer letras. En general, cuando aparecen letras en las claves se suele asociar a cada letra un entero, o se utiliza su código ASCII.

Existen otras funciones de randomización o asignación, las cuales tienen sus ventajas y desventajas dependiendo del conjunto de claves a las que se ha de aplicar la función de asignación.

Una consideración muy importante cuando se elige una función Hash es la *eficiencia de cálculo*, no es suficiente que se pueda encontrar el elemento en el primer intento si este intento toma mucho más tiempo que otros.

### 4.2.- Soluciones al problema de las colisiones

Supongamos que queremos añadir un nuevo elemento  $k$ , pero supongamos que la posición de memoria  $H(k)$  ya está ocupada. Esta situación se llama *colisión*.

A continuación se presentan las formas generales de resolver las colisiones:

- Rehashing o Reasignación.
- Arreglos anidados o Cubos.
- Encadenamiento o Tablas Hash Abiertas.
- Zona de Desbordamiento.

El procedimiento particular que se escoja dependerá de muchos factores. Un factor importante es la relación entre el número  $n$  de elementos y el número  $m$  de tamaño del arreglo. Esta razón,  $\lambda = n/m$  se llama *factor de carga*.

Las colisiones son casi imposibles de evitar. Véase el siguiente ejemplo:

Si una clase tiene 24 alumnos y tenemos un arreglo con espacio para 365 registros. Una función hash aleatoria es la de escoger la fecha de nacimiento como dirección hash. Aunque el factor de carga  $\lambda = 24/365$  (aproximadamente el 7%) es muy pequeño, se puede demostrar que existe una posibilidad mayor del 50% de que dos alumnos hayan nacido el mismo día.

La *eficiencia* de una función hash con un procedimiento de resolución de colisiones se mide por el número medio de *pruebas* (comparaciones entre elementos) necesarias para encontrar la posición de un elemento  $X$  dado. La eficiencia depende principalmente del factor de carga  $\lambda$ . En concreto, interesa conocer:

$S(\lambda)$  = número medio de celdas examinadas para una búsqueda CON éxito.

$U(\lambda)$  = número medio de celdas examinadas para una búsqueda SIN éxito.

#### 4.2.1.- Rehashing o Reasignación

Se trata de buscar (o colocar, ya que los datos que pretenden ser localizados en el arreglo mediante su dirección hash, *deberán ser previamente almacenados utilizando esa misma dirección*) el elemento cuya posición ya está ocupada, en otra posición disponible en el arreglo. Esto se hace mediante la función de *reasignación*  $R(H(X))$ , la cual acepta un índice del arreglo y produce otro.

Si la posición del arreglo dada por  $H(X)$  se encuentra ocupada (o, en caso de búsqueda, ocupada por un elemento distinto al buscado), se aplica la función  $R$  sobre  $H(X)$  para encontrar otra posición donde se pueda colocar (o localizar) el elemento. Si de nuevo  $R(H(X))$  se encuentra también ocupada, se vuelve a aplicar

## Apunte de Cátedra

rehashing hasta encontrar una posición vacía, en cuyo caso se realizaría la inserción (o en caso de tratarse de una búsqueda, indicaría que el elemento buscado no existe).

Existen varios métodos que trabajan bajo el principio de comparación y reasignación de elementos:

- Prueba lineal o secuencial.
- Prueba cuadrática.
- Doble dirección hash.

### 4.2.1.1.- Prueba Lineal o Secuencial

Consiste en que una vez detectada la colisión se debe recorrer el arreglo secuencialmente a partir del punto de colisión, buscando al elemento. El proceso de búsqueda concluye cuando el elemento es hallado, o bien cuando se encuentra una posición vacía. Se trata al arreglo como a una estructura circular: el siguiente elemento después del último es el primero. La función de rehashing es, por lo tanto, de la forma:

$$R(H(X)) = H(X) + 1$$

Ejemplo:

Si la posición 397 ya estaba ocupada, el registro con clave 0596397 es colocado en la posición 398, la cual se encuentra disponible. Una vez que el registro ha sido insertado en esta posición, otro registro que genere la posición 397 o la 398 es insertado en la posición siguiente disponible.

Una implementación en Java de búsqueda hash solucionando las colisiones por medio de la prueba lineal o secuencial se muestra a continuación:

```
public class Hashing
{
    public int[] table;

    public Hashing(int count)
    {
        table=new int[count];
    }

    public void addSequential(int value)
    {
        int place=getHash(value);
        if(table[place]!=0)
        {
            do
                place=getNextPlace(place);
            while(table[place]!=0);
        }
        table[place]=value;
    }

    public int searchSequential(int search)
    {
        int place=getHash(search);
        if(table[place]!=search)
        {
            //para que no siga en forma circular infinitamente
            int init=place;
            do
                place=getNextPlace(place);
            while(table[place]!=search & init!=place);
            if(init==place)
                return -1;
        }
        return place;
    }
}
```

## Apunte de Cátedra

```
private int getHash(int value)
{
    return value % table.length;
}

private int getNextPlace(int place)
{
    place++;
    if(place == table.length)
        place=0;
    return place;
}
}
```

Utilizando este método de resolución de colisiones, el número medio de pruebas para una búsqueda CON éxito es:

$$S(\lambda) = 1/2 (1 + (1 / (1 - \lambda)))$$

y el número medio de pruebas para una búsqueda SIN éxito es:

$$U(\lambda) = 1/2 (1 + (1 / (1 - \lambda)^2))$$

La principal desventaja de este método es que puede haber un fuerte agrupamiento alrededor de ciertas claves, mientras que otras zonas del arreglo permanezcan vacías. Si las concentraciones de claves son muy frecuentes, la búsqueda será principalmente secuencial perdiendo así las ventajas del método hash. Por lo tanto, una propiedad de una buena función de reasignación es que *para cualquier índice i, las reasignaciones sucesivas  $R(H(i))$ ,  $R(H(R(H(i))))$ ,... cubran la mayoría de los enteros entre 0 y la longitud del arreglo.*

Se cumple que, para cualquier función  $R(H(i)) = (i \% m) + c$ , donde  $m$  es el número de elementos del arreglo y  $c$  es una constante tal que  $c$  y  $m$  son *primos relativos* (es decir, no tienen factores en común), *genera valores sucesivos que cubren todo el arreglo.* Si  $m$  y  $c$  tienen factores en común, el número de posiciones diferentes del arreglo que se obtienen será el cociente de dividir  $m$  y  $c$ .

Basándonos en esta última afirmación, el algoritmo de búsqueda (y el de carga) tienen un problema: utilizando una función de rehashing de este tipo podrían salir sin inserción aun cuando exista alguna posición vacía en el arreglo.

Ejemplo:

Dada la función de reasignación:

$$RH(i) = (i \% 1000) + 200$$

Con esta función, cada clave solo puede ser colocada en cinco posiciones posibles:  $(1000/200=5)$  si  $i = 215$ , y esta posición está ocupada, entonces se reasigna de la siguiente forma:

$$RH(215) = (215 \% 1000) + 200 = 415$$

$$RH(415) = (415 \% 1000) + 200 = 615$$

$$RH(615) = (615 \% 1000) + 200 = 815$$

$$RH(815) = (815 \% 1000) + 200 = 15$$

$$RH(15) = (15 \% 1000) + 200 = 215$$

$$RH(215) = (215 \% 1000) + 200 = 415$$

...

Si estas cinco posiciones posibles están ocupadas, saldremos sin inserción aun cuando haya posiciones libres en el arreglo.

### Colisión por rehashing: agrupamiento o clustering

A pesar de la utilización de una función de rehashing donde  $c$  y  $m$  sean primos, con el método de prueba lineal o secuencial los elementos siguen tendiendo a *agruparse*, o sea, a aparecer unos junto a otros, cuando el factor de carga es mayor del 50%. Cuando el arreglo está vacío es igualmente probable que cualquier

## Apunte de Cátedra

elemento al azar sea colocado en cualquier posición libre dentro del arreglo, pero una vez que se han tenido algunas entradas y se han presentado varias colisiones o choques en la asignación, esto no es cierto.

Ejemplo:

| Claves  |
|---------|
| 4618396 |
| 4957397 |
|         |
| 1286399 |
|         |
|         |
| 0000990 |
| 0000991 |
| 1200992 |
| 0047993 |
|         |
| 9846995 |

En este caso, es cinco veces más probable que un registro sea insertado en la posición 994 que en la posición 401. Esto se debe a que cualquier registro cuya clave genera la asignación 990, 991, 992, 993 o 994 será colocado en la 994, mientras que cualquier registro cuya clave genera la asignación 401 será colocado en su posición.

Este fenómeno en el que *dos claves que inicialmente generan una asignación en dos sitios diferentes, luego compiten entre sí en reasignaciones sucesivas*, se denomina *agrupamiento o clustering*:  $H(X) \neq H(Y)$  pero  $RH(X) = RH(Y)$  en reasignaciones sucesivas.

Se cumple que:

- cualquier función de reasignación que dependa únicamente del índice dará lugar a agrupamiento.
- daría agrupamiento la función de reasignación  $Rh(i) = (i \% m) + c$ , aunque  $c$  y  $m$  fuesen primos.

Ejemplo:

Si  $m=1000$ ,  $c=21$  y las posiciones 10, 31, 52, 73 y 94 están todas ocupadas, cualquier elemento cuya clave sea cualquiera de estos cinco enteros sería colocado en la posición 115. Se dice entonces que 10, 31, 52, 73 y 94 forman un *cluster*.

El clustering se debe a que las reasignaciones sucesivas siguen siempre la misma secuencia. Si conseguimos que esa secuencia varíe en cada reasignación se evitaría la formación del cluster.

Las técnicas que se presentan a continuación minimizan el agrupamiento:

### 4.2.1.2.- Prueba Cuadrática

Este método es similar al de la prueba lineal o secuencial. La diferencia consiste en que, en lugar de buscar en las posiciones con direcciones:  $dirHash$ ,  $dirHash + 1$ ,  $dirHash + 2$ ,  $dirHash + 3$ , ... buscamos linealmente en las posiciones con direcciones:  $dirHash$ ,  $dirHash + 1$ ,  $dirHash + 4$ ,  $dirHash + 9$ , ...,  $dirHash + i^2$ .

Esta variación permite una mejor distribución de las claves colisionadas.

Si el número  $m$  de posiciones en el arreglo  $T$  es un número primo y el factor de carga no excede del 50%, sabemos que siempre insertaremos el nuevo elemento  $X$  y que ninguna celda será consultada dos veces durante un acceso.

## Apunte de Cátedra

### 4.2.1.3.- Doble Direccionamiento Hash

Consiste en que una vez detectada la colisión se debe generar otra dirección aplicando una función hash  $H_2$  a la dirección previamente obtenida. Entonces buscamos linealmente en las posiciones que se encuentran a una distancia  $H_2(X)$ ,  $2 H_2(X)$ ,  $3 H_2(X)$ , ...

La función hash  $H_2$  que se aplique a las sucesivas direcciones puede ser o no ser la misma que originalmente se aplicó a la clave. No existe una regla que permita decidir cuál será la mejor función a emplear en el cálculo de las sucesivas direcciones. Pero una buena elección sería  $H_2(X) = R - (X \% R)$ , siendo  $R$  un número primo más pequeño que el tamaño del arreglo. Y se obtienen resultados mejores cuando el tamaño del arreglo es un número primo.

### Problemas de la técnica de rehashing

- Inserción: como se asume un arreglo de tamaño fijo, si el número de elementos aumenta más allá de ese tamaño es imposible insertarlo sin que sea necesario asignar un arreglo más grande y recalcular los valores de asignación de las claves de todos los elementos que ya se encuentran en el arreglo utilizando una nueva función de asignación.
- Borrado: es difícil eliminar un elemento. Por ejemplo, si el elemento  $r1$  está en la posición  $p$ , para añadir un elemento  $r2$  cuya clave  $k2$  queda asignada en  $p$ , éste debe ser insertado en la primera posición libre de las siguientes:  $Rh(p)$ ,  $Rh(Rh(p))$ ... Si luego  $r1$  es eliminado y la posición  $p$  queda vacía, una búsqueda posterior del elemento  $r2$  comenzará en la posición  $H(k2) = p$ . Como esta posición está ahora vacía, el proceso de búsqueda puede erróneamente llevarnos a la conclusión de que el elemento  $r2$  no se encuentra en el arreglo.

Una posible solución a este problema es marcar el elemento eliminado como 'eliminado' en vez de 'vacío', y continuar la búsqueda cuando se encuentra una posición como 'eliminada'. Pero esto sólo es factible para un número pequeño de eliminaciones pues, en caso contrario, una búsqueda sin éxito requerirá una búsqueda a través de todo el arreglo debido a que la mayoría de las posiciones estarán marcadas como 'eliminado', en lugar de 'vacío'. Por esto, el direccionamiento directo normalmente no se usará cuando el arreglo vaya a ser modificado constantemente.

### 4.2.2.- Arreglos anidados o Cubos

Este método consiste en que cada elemento del arreglo tenga otro arreglo en el cual se almacenen los elementos colisionados. Si bien la solución parece ser sencilla, es claro también que resulta ineficiente. Al trabajar con arreglos se depende del espacio que se le haya asignado a éste. Lo cual conduce a un nuevo problema difícil de solucionar: elegir un tamaño adecuado de arreglo que permita un equilibrio entre el costo de memoria y el número de valores colisionados que pudiera almacenar.

Cuando el cubo se llena, debemos tratar de nuevo con colisiones.

Existen dos maneras de implementarlo, una sería usando arreglos anidados (arreglo de arreglos) y la otra sería usando matrices (cubos). Vemos esta última implementación con un ejemplo:

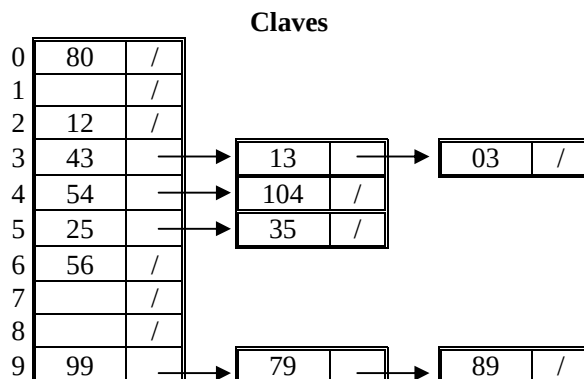
| Claves |    |     |    |     |
|--------|----|-----|----|-----|
| 0      | 80 |     |    | ... |
| 1      |    |     |    | ... |
| 2      | 12 |     |    | ... |
| 3      | 43 | 13  | 03 | ... |
| 4      | 54 | 104 |    | ... |
| 5      | 25 | 35  |    | ... |
| 6      | 56 |     |    | ... |
| 7      |    |     |    | ... |
| 8      |    |     |    | ... |
| 9      | 99 | 79  | 89 | ... |

## Apunte de Cátedra

### 4.2.3.- Encadenamiento o Tablas Hash Abiertas

Otro método para resolver las colisiones consiste en mantener una lista encadenada de todos los elementos cuyas claves generan la misma posición. Si la función hash genera valores entre 0 y  $(m - 1)$ , declaramos un arreglo de nodos de encabezamiento de tamaño  $m$ , de manera que  $\text{Tabla}[i]$  apunta a la lista con todos los elementos cuyas claves generan posiciones en  $i$ .

Ejemplo:



- **Inserción:** al buscar un elemento cuya función hash le hace corresponder la posición  $i$ , se accede a la cabeza de la lista correspondiente a esa posición:  $\text{Tabla}[i]$ , y se recorre la lista que dicha posición inicia. Si éste no se encuentra entonces se inserta al final de la lista.
- **Eliminación:** la eliminación de un nodo de un arreglo que ha sido construida mediante randomización y encadenamiento se reduce simplemente a remover un nodo de la lista encadenada. Un nodo eliminado no afecta a la eficiencia del algoritmo de búsqueda. El algoritmo continúa como si el nodo nunca se hubiera insertado.

Estas listas pueden reorganizarse para maximizar la eficiencia de búsqueda, utilizando diferentes métodos:

**Probabilidad.** Consiste en insertar los elementos dentro de la lista en su punto apropiado. Esto significa que si  $\text{prob}$  es la probabilidad de que un elemento sea el argumento buscado, entonces el elemento debe insertarse entre los elementos  $r(i)$  y  $r(i + 1)$ , donde  $i$  es tal que:

$$P(i) \geq \text{prob} \geq P(i + 1)$$

El problema es que  $P(i)$  pocas veces se conocen a priori, y aunque algunos elementos sean buscados más frecuentemente que otros, es casi imposible identificar dichos elementos por adelantado. Además, la probabilidad de que un elemento determinado sea recuperado puede cambiar con el tiempo.

**Movimiento al frente.** Cuando una búsqueda ha tenido éxito, el elemento encontrado es retirado de su posición actual en la lista y colocado en la cabeza de dicha lista.

**Trasposición.** Cuando una búsqueda de un elemento ha tenido éxito es intercambiado con el elemento que le precede inmediatamente, de manera que si es accedido muchas veces llegará a ocupar la primera posición de la lista.

El método de trasposición adelantando solo una posición evita el inconveniente que tiene el de movimiento al frente, que solamente porque un elemento sea recuperado una vez es movido inmediatamente al frente de la lista, reduciendo la eficiencia de la búsqueda con respecto a otros elementos que le precedían. El método de trasposición asegura que al avanzar un elemento en una posición cada vez que es obtenido, éste sólo avanzará hasta el frente de la lista si es sacado con una alta frecuencia.

Otra ventaja del método de trasposición sobre el de movimiento al frente es que se puede aplicar eficientemente tanto a arreglos de arreglos como a arreglos de listas. La trasposición de dos elementos en un arreglo es una operación relativamente eficiente, mientras que el movimiento desde el medio de un arreglo hasta el frente implica (en promedio) mover medio arreglo.

## Apunte de Cátedra

El número medio de pruebas, con encadenamiento, para una búsqueda con éxito y para una búsqueda sin éxito tiene los siguientes valores aproximados:

$$S(\lambda) = 1 + \frac{1}{2} \lambda$$

$$U(\lambda) = e^{-\lambda} + \lambda$$

### Ventajas y desventajas del uso del encadenamiento

- Ventajas: es el método más eficiente debido al dinamismo propio de las listas. Cualquiera que sea el número de colisiones registradas en una posición, siempre será posible tratar una más.
- Desventajas: la desventaja principal es el espacio extra adicional que se requiere para la referencia a otros elementos. Sin embargo, el arreglo inicial es generalmente pequeño en esquemas que utilizan encadenamiento comparado con aquellos que utilizan reasignación. Esto se debe a que el encadenamiento es menos catastrófico si el arreglo llega a llenarse completamente, pues siempre es posible asignar más nodos y añadirlos a varias listas.

Por supuesto, si las listas llegan a ser muy largas no se va a obtener ninguno de los beneficios de la randomización o hashing, como eran el direccionamiento directo y la eficiencia de búsqueda resultante.

### 4.2.4.- Zona de Desbordamiento

Se trata de mantener una zona reservada para aquellos elementos que llegan a colisionar, de manera que cuando se produzca una colisión el elemento se va a localizar en esta zona de desbordamiento.

Al realizar la búsqueda y comprobar si el elemento buscado está en la posición dada por su tabla hash, si esa posición ya está ocupada por otro elemento con el mismo valor de hashing, se seguirá buscando a partir del inicio de la zona de desbordamiento de manera secuencial, hasta encontrar el elemento o llegar al final de dicha zona de desbordamiento.

Ejemplo:

| Claves |    | Desbordamientos |     |
|--------|----|-----------------|-----|
| 0      | 80 | 0               | 13  |
| 1      |    | 1               | 104 |
| 2      | 12 | 2               | 79  |
| 3      | 43 | 3               | 03  |
| 4      | 54 | 4               | 35  |
| 5      | 25 | 5               | 89  |
| 6      | 56 | .               |     |
| 7      |    | .               |     |
| 8      |    | .               |     |
| 9      | 99 | n               |     |